S.Y. B. Tech. Academic Year 2026_27 Semester: III

Data Structures

LABORATORY WRITE UP

Experiment Number: 05

TITLE: Circular Linked List Operations

AIM:

Write a program to implement polynomial operations using a Circular Linked List. The program should support the following operations:

1. Create a polynomial by inserting terms (coefficient and exponent).
2. Display the polynomial in a readable mathematical form.
3. Add two polynomials and display the resultant polynomial.

OBJECTIVES:

1. **To implement a polynomial representation using a Circular Linked List** by dynamically creating and storing polynomial terms (coefficient and exponent).
2. **To perform fundamental polynomial operations** such as creating, displaying, and adding two polynomials using Circular Linked List techniques.
3. **To understand the advantages of Circular Linked Lists** in managing dynamic data structures and efficiently handling polynomial arithmetic by combining like terms.

THEORY:

A polynomial is a mathematical expression made up of terms, where each term consists of a coefficient and a variable raised to a non-negative integer exponent. A general polynomial can be represented as:

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

where a_n, a_{n-1},…,a_0 are coefficients and n is the highest exponent of the polynomial. For example:

$$P(x) = 5x^3 + 2x^2 - 4x + 7$$

In a computer program, each polynomial term can be represented using two fields: coefficient and exponent. Since the number of terms in a polynomial may vary, a dynamic data structure such as a linked list is suitable for storing polynomial terms.

A Circular Linked List is a variation of a linked list in which the last node points back to the first node instead of containing a NULL pointer. This creates a circular structure. Each node in the circular linked list contains the coefficient, exponent, and a pointer to the next node. The circular nature of the list is useful for repeated traversal and processing of polynomial terms without requiring a separate NULL condition.

For polynomial representation, the terms are generally stored in descending order of exponents. For example, the polynomial:

$$6x^4 + 3x^2 + 2x + 1$$

can be stored as:

$$(6, 4) \rightarrow (3, 2) \rightarrow (2, 1) \rightarrow (1, 0)$$

where each pair represents (coefficient, exponent). The last node is linked back to the first node, forming a circular linked list.

The creation operation inserts polynomial terms into the circular linked list. While inserting, the program should maintain the order of exponents and avoid storing unnecessary terms with a coefficient of zero. If a term with an existing exponent is inserted, its coefficient can be combined with the existing term.

The display operation traverses the circular linked list and prints each term in a readable mathematical format. Special cases such as exponent zero, exponent one, positive and negative coefficients, and coefficients equal to one must be handled properly. For example, a node containing coefficient 3 and exponent 2 is displayed as 3x^2, while a node containing coefficient 5 and exponent 0 is displayed as 5.

Polynomial addition is performed by comparing the exponents of terms from two polynomials. If the exponents are equal, the coefficients are added and the resulting term is inserted into the resultant polynomial. If the exponent of one term is greater than the other, that term is directly copied to the resultant polynomial. This process continues until all terms from both polynomials are processed. Terms having a resultant coefficient of zero are not included in the final polynomial.

IMPLEMENTATION:

- **PLATFORM:**

- 64-bit Open source Linux or its derivatives.
- Open Source C Programming tool like gcc/Eclipse Editor.

- **TEST CONDITIONS:-**

1. Implement a polynomial representation using a circular linked list
   - Create a polynomial with multiple terms entered in sequence
   - Create a Polynomial with a single term
   - Create an empty polynomial with 0 terms
   - Verify that the last node points back to the first node
2. Perform polynomial operations (Create, Display, Addition)
   - Display a polynomial with multiple terms.
   - Add two polynomials having common exponents.
   - Add two polynomials having no common exponents
   - Add polynomials with negative coefficients
3. Understand efficient handling of polynomial arithmetic using Circular linked list
   - Perform repeated traversal of the polynomial
   - Verify memory allocation for dynamic node creation.
   - Display the resultant polynomial after addition

- **PSEUDO CODE:** ***//To be Written by Students***

  Write pseudo code for create_polynomial, display_polynomial, addPolynomial using Circular Linked List.

- **TIME COMPLEXITY:** ***//To be Written by Students***

  Write time complexity of above operations

- **CONCLUSION:**

The program successfully implements polynomial creation, display, and addition using a Circular Linked List, demonstrating efficient dynamic memory management and polynomial manipulation.

- **FAQs** ***//To be Written by Students***

  - Why is a Circular Linked List used to represent polynomials instead of a Linear Linked List?
  - What information is stored in each node of the Circular Linked List?

- What happens if one of the polynomials is empty during the addition operation?

- **PRACTICE ASSIGNMENTS**

1. Implement a program to evaluate a polynomial represented using a **Circular Linked List** for a given value of x. The program should allow the user to create a polynomial, display it, input a value of x, and compute the polynomial's numerical value.

2. Implement polynomial multiplication using a **Circular Linked List**. Create two polynomials, multiply them term-by-term, combine like terms, and display the resultant polynomial in standard mathematical form.